

Name : Farukh Hasan Shaha Div: SY - B Roll No. : B - 35 Branch: AI&D

AIM: Implementing Web Scraping in Python with BeautifulSoup. Theory : 1.What is web scraping? Explain the role of the Requests library in web scraping. ANS : Web scraping is the process of automatically extracting data from websites. The Requests library is used to:- Send HTTP requests (GET, POST) to a website -Fetch the webpage content (HTML, JSON, etc.) -Act like a browser to access web data 2.What is BeautifulSoup? How does it help in parsing HTML? ANS : BeautifulSoup is a Python library used to:-Parse HTML and XML documents -Extract specific data from web pages It converts messy HTML into a structured tree format, so you can easily:-Find tags (, etc.) -Extract text, links, tables 3.Differentiate between web scraping and web crawling. ANS : Web scraping and web crawling are related but different processes. Web scraping focuses on extracting specific data from websites. It targets particular elements such as product prices, headlines, or contact information from a webpage. On the other hand, web crawling is the process of systematically browsing the web to discover and index URLs. It is commonly used by search engines to explore web pages and build an index. In simple terms, web crawling is about finding web pages, while web scraping is about extracting data from those pages. 4.What are the commonly used methods in BeautifulSoup? ANS : Some important methods include: find() → Finds first matching tag find_all() → Finds all matching tags get_text() → Extracts text get() → Gets attribute (like href) select() → Uses CSS selectors prettify() → Formats HTML nicely 5.What are the ethical and legal considerations in web scraping? ANS : Ethical rules:-Respect website terms and conditions - Avoid overloading servers (use delays) -Do not scrape private or sensitive data Legal rules:-Follow copyright laws -Check website permissions (robots.txt) -Avoid scraping

```
import requests
from bs4 import BeautifulSoup
import csv

# Step 1: Define URL
url = "https://books.toscrape.com/"

# Step 2: Send HTTP GET request
response = requests.get(url)

# Step 3: Check status
if response.status_code == 200:
    print("Successfully fetched the webpage")
else:
    print(f"Failed. Status code: {response.status_code}")
    exit()

# Step 4: Parse HTML
soup = BeautifulSoup(response.text, 'html.parser')

# Step 5: Find 5 book containers
# Each book is inside <article class="product_pod">
books = soup.find_all('article', class_='product_pod')[:5]
print(f"\nTotal books found: {len(books)}")
print("=" * 55)

# Step 6: Extract data from each book
data = []
for index, book in enumerate(books, start=1):
    # Extract BOOK TITLE
    title = book.find('h3').find('a')['title']
    # Extract PRICE
    price = book.find('p', class_='price_color').get_text()
    # Extract RATING
    rating_tag = book.find('p', class_='star-rating')
    rating = rating_tag['class'][1] # Gets: One, Two, Three, Four, Five
    # Extract AVAILABILITY
    availability = book.find('p', class_='instock availability').get_text(strip=True)

    print(f"\nBook {index}:")
    print(f" Title : {title}")
    print(f" Price : {price}")
    print(f" Rating : {rating} Star")
    print(f" Availability : {availability}")
    data.append({
        'Book No' : index,
        'Title' : title,
        'Price' : price,
        'Rating' : rating + " Star",
        'Availability' : availability
    })

# Step 7: Save to CSV
filename = "books.csv"
with open(filename, mode='w', newline='', encoding='utf-8') as file:
    fieldnames = ['Book No', 'Title', 'Price', 'Rating', 'Availability']
```

```

writer = csv.DictWriter(file, fieldnames=fieldnames)
writer.writeheader()
for row in data:
    writer.writerow(row)
print(f"\n{' ' * 55}")
print(f"Data saved to '{filename}' successfully!")
print(f"Total books saved: {len(data)}")

# Step 8: Read back CSV to verify
print("\n— Verifying CSV Contents —")
print("=" * 55)
with open(filename, mode='r', encoding='utf-8') as file:
    reader = csv.DictReader(file)
    for row in reader:
        print(f"\nBook {row['Book No']}:")
        print(f"  Title : {row['Title']}")
        print(f"  Price : {row['Price']}")
        print(f"  Rating : {row['Rating']}")
        print(f"  Availability : {row['Availability']}")

```

Successfully fetched the webpage

Total books found: 5

=====

Book 1:

Title : A Light in the Attic
 Price : £51.77
 Rating : Three Star
 Availability : In stock

Book 2:

Title : Tipping the Velvet
 Price : £53.74
 Rating : One Star
 Availability : In stock

Book 3:

Title : Soumission
 Price : £50.10
 Rating : One Star
 Availability : In stock

Book 4:

Title : Sharp Objects
 Price : £47.82
 Rating : Four Star
 Availability : In stock

Book 5:

Title : Sapiens: A Brief History of Humankind
 Price : £54.23
 Rating : Five Star
 Availability : In stock

=====

Data saved to 'books.csv' successfully!

Total books saved: 5

— Verifying CSV Contents —

=====

Book 1:

Title : A Light in the Attic
 Price : £51.77
 Rating : Three Star
 Availability : In stock

Book 2:

Title : Tipping the Velvet
 Price : £53.74
 Rating : One Star
 Availability : In stock

Book 3:

Title : Soumission
 Price : £50.10
 Rating : One Star

— PART 2: Generic Scraper (User Input) + Save to CSV —

```

import requests
from bs4 import BeautifulSoup
import csv

# Step 1: Take inputs from user
url = input("Enter the URL to scrape: ")
tag = input("Enter HTML tag (e.g., p, h1, div): ")
class_name = input("Enter class name (blank if none): ")

```

```

# Step 2: Fetch the webpage with error handling
try:
    response = requests.get(url, timeout=10)
    if response.status_code != 200:
        print("Failed. Status code:", response.status_code)
        exit()
except Exception as e:
    print("Error:", e)
    exit()

# Step 3: Parse HTML
soup = BeautifulSoup(response.text, 'html.parser')
# Step 4: Find elements – with class or without
if class_name.strip():
    elements = soup.find_all(tag, class_=class_name.strip())
else:
    elements = soup.find_all(tag)
# Step 5: Display results + store in list
if not elements:
    print("No elements found.")
else:
    print(f"\nFound {len(elements)} element(s):\n")
    data = []
    for i, el in enumerate(elements, 1):
        text = el.get_text(strip=True)
        attrs = str(el.attrs)
        print(f"Element {i}:")
        print(" Text :", text)
        print(" Attributes:", attrs)
        print("-" * 50)
        data.append({
            'Element No' : i,
            'Tag' : tag,
            'Text' : text,
            'Attributes' : attrs
        })

# Step 6: Print page title
title = soup.title.string if soup.title else "N/A"
print("\nPage Title:", title)
# Step 7: Save to CSV file
if data:
    filename = "scraped_data.csv"
    with open(filename, mode='w', newline='', encoding='utf-8') as file:
        fieldnames = ['Element No', 'Tag', 'Text', 'Attributes']
        writer = csv.DictWriter(file, fieldnames=fieldnames)
        writer.writeheader()
        for row in data:
            writer.writerow(row)
    print(f"\nData saved to '{filename}' successfully!")
    print(f"Total rows written: {len(data)}")
print("-" * 50)
# Step 8: Read back CSV to verify
print("\n— Verifying CSV Contents —")
print("-" * 50)
with open(filename, mode='r', encoding='utf-8') as file:
    reader = csv.DictReader(file)
    for row in reader:
        print(f"\nElement {row['Element No']}:")
        print(f" Tag : {row['Tag']}")
        print(f" Text : {row['Text']}")
        print(f" Attributes : {row['Attributes']}")

```

Enter the URL to scrape: <https://www.geeksforgeeks.org>

Enter HTML tag (e.g., p, h1, div): p

Enter class name (blank if none):

Found 1 element(s):

Element 1:

Text : Interested in advertising with us?

Attributes: {'class': ['HomePageAdvertiseWithUsCard_advertiseWithUsCard_textContainer--mainText__zIsPu']}

Page Title: GeeksforGeeks | Your All-in-One Learning Portal

Data saved to 'scraped_data.csv' successfully!

Total rows written: 1

— Verifying CSV Contents —

Element 1:

Tag : p

```
Text : Interested in advertising with us?  
Attributes : {'class': ['HomePageAdvertiseWithUsCard_advertiseWithUsCard_textContainer--mainText__zIsPu']}
```

Conclusion : Web scraping using Python was successfully implemented with the help of the Requests and BeautifulSoup libraries. The program was able to fetch HTML content from a given website and extract specific data based on user inputs such as URL, HTML tag, and class name. It was observed that static websites provide accurate and complete data, while dynamic websites (like the homepage of GeeksforGeeks) may return limited results because their content is loaded using JavaScript. The extracted data was displayed and also stored in a CSV file for further use. This experiment helped in understanding how web pages are structured and how meaningful information can be extracted efficiently. Overall, web scraping is a powerful technique for data collection, but it must be performed responsibly by following ethical and legal guidelines